Linux • UFW • NAT/PAT • WireGuard

Лекция 9. Linux-router, PAT, Port Forwarding и WireGuard VPN

Маршрутизация, трансляция портов, публикация сервисов и удалённый доступ

Цель лекции

- Понять, как Linux-сервер может работать маршрутизатором, выпускать LAN наружу через PAT, публиковать внутренние сервисы и принимать удалённых пользователей через WireGuard.
- Объяснять роль Linux-router между WAN и LAN.
- Включать IPv4 forwarding и понимать транзитный трафик.
- Понимать, как PAT позволяет многим внутренним клиентам использовать один внешний IP.
- Отличать исходящий PAT от входящего Port Forwarding.
- Понимать назначение Remote Access VPN.
- Объяснять WireGuard через ключи, peers, Endpoint и AllowedIPs.
- Диагностировать маршруты, PAT, UFW и WireGuard.

Учебные вопросы

- 1. Linux как маршрутизатор корпоративной сети.
- 2. NAT и PAT: выход LAN наружу через один внешний адрес.
- 3. Port Forwarding: публикация внутреннего сервиса.
- 4. VPN: удалённый доступ и объединение сетей.
- 5. WireGuard: peers, ключи, Endpoint и AllowedIPs.
- 6. Практика: базовая настройка WireGuard.
- 7. Диагностика Linux-router, PAT и WireGuard.

Учебный сценарий

- gw01.company.test работает как Linux-router между внешней и внутренней сетью.
- WAN-интерфейс enp0s3: учебный внешний адрес 203.0.113.10.
- LAN-интерфейс enp0s8: 192.168.10.1/24.
- client01 в LAN: 192.168.10.10.
- web01 в LAN: 192.168.10.20.
- VPN-интерфейс wg0: 10.8.0.1/24.
- Удалённый клиент WireGuard: 10.8.0.2/32.

Адресный план

Узел / интерфейс	Адрес	Назначение
gw01 enp0s3	203.0.113.10	WAN / внешняя сеть
gw01 enp0s8	192.168.10.1/24	LAN gateway
client01	192.168.10.10/24	Клиент LAN
web01	192.168.10.20/24	Внутренний web-сервер
gw01 wg0	10.8.0.1/24	VPN-сервер
laptop01 wg0	10.8.0.2/32	VPN-клиент

1. Linux как маршрутизатор

Интерфейсы, маршруты, gateway и ip_forward

Что делает маршрутизатор

- Маршрутизатор пересылает IP-пакеты между разными сетями.
- У gw01 есть минимум два интерфейса: WAN и LAN.
- LAN-клиенты отправляют пакеты на gateway 192.168.10.1.
- gw01 решает, через какой интерфейс отправить пакет дальше.
- Для транзитного трафика нужен включённый IPv4 forwarding.
- Для выхода частной LAN наружу дополнительно нужен PAT.

Интерфейсы gw01

- enp0s3 — внешний интерфейс в сторону WAN.
- enp0s8 — внутренний интерфейс в сторону LAN.
- 203.0.113.10 — учебный внешний адрес из документационного диапазона.
- 192.168.10.1/24 — адрес gateway для LAN.
- У клиентов LAN default gateway должен быть 192.168.10.1.

Что такое транзитный пакет

- Транзитный пакет не предназначен самому gw01: он должен пройти через маршрутизатор в другую сеть.
- client01 отправляет пакет к внешнему адресу.
- Пакет приходит на enp0s8.
- gw01 смотрит таблицу маршрутизации.
- Если маршрут найден, пакет должен выйти через другой интерфейс.
- Если IPv4 forwarding выключен, Linux не пересылает пакет дальше.

Включение IPv4 forwarding

BASH

```
sudo sysctl -w net.ipv4.ip_forward=1

echo 'net.ipv4.ip_forward=1' | \
  sudo tee /etc/sysctl.d/99-router.conf

sudo sysctl --system
sysctl net.ipv4.ip_forward
```

Что важно

- sysctl -w включает пересылку до перезагрузки.
- Файл в /etc/sysctl.d делает настройку постоянной.
- sysctl --system перечитывает системные параметры.
- Ожидаемое значение: net.ipv4.ip_forward = 1.

Включение IPv4 forwarding альтернативный способ

BASH

```
# Создать файл настройки:
sudo nano /etc/sysctl.d/99-forwarding.conf

# Вставить в него строку:
net.ipv4.ip_forward=1

# Сохранить файл:
   Ctrl+O → Enter → Ctrl+X

# Применить настройку:
sudo sysctl --system

# Проверить:
sudo sysctl net.ipv4.ip_forward

# Должно вывести:
net.ipv4.ip_forward = 1
```

Что важно

- именно 99-forwarding.conf называть не обязательно.
- Важно только, чтобы файл:

- находился в /etc/sysctl.d/;
- имел расширение .conf.

Включение IPv6 forwarding

BASH

```
# В тот же файл настройки:
sudo nano /etc/sysctl.d/99-forwarding.conf

# Вставить строку:
net.ipv6.conf.all.forwarding=1

# Сохранить файл:
   Ctrl+O → Enter → Ctrl+X

# Применить настройку:
sudo sysctl --system

# Проверить:
sysctl net.ipv6.conf.all.forwarding

# Должно вывести:
net.ipv6.conf.all.forwarding = 1
```

Что важно

- IPv4 и IPv6 маршрутизация включаются двумя строками в /etc/sysctl.d/99-forwarding.conf

Таблица маршрутизации

- Таблица маршрутизации определяет, куда отправить пакет.
- Команда ip route показывает маршруты Linux-узла.
- Более точный маршрут имеет приоритет над менее точным.
- default route используется, если нет более точного совпадения.
- На gw01 default route должен вести во внешнюю сеть.
- На client01 default route должен вести на 192.168.10.1.

Пример маршрутов gw01

BASH

```
ip route

# Пример:
default via 203.0.113.1 dev enp0s3
203.0.113.0/24 dev enp0s3 proto kernel scope link
192.168.10.0/24 dev enp0s8 proto kernel scope link
```

Что важно

- default via 203.0.113.1 — путь во внешнюю сеть.
- 192.168.10.0/24 dev enp0s8 — путь в LAN.
- Для локально подключённых сетей маршруты создаются автоматически.
- Если default route неверный, выход наружу не заработает.

Пример маршрутов client01

BASH

```
ip route

# Пример:
default via 192.168.10.1 dev enp0s3
192.168.10.0/24 dev enp0s3 proto kernel scope link
```

Что важно

- client01 знает свою локальную сеть 192.168.10.0/24.
- Для всех остальных сетей он использует gateway 192.168.10.1.
- Если gateway не указан, client01 не сможет отправлять пакеты наружу.
- Если gateway указан неверно, ответов от внешних узлов не будет.

Прохождение пакета через Linux-router

Как транзитный пакет проходит через маршрутизатор, firewall и NAT

LAN 192.168.10.0/24

client01



192.168.10.10

Linux-router gw01

enp0s8
192.168.10.1/24



enp0s3
203.0.113.10

1. Клиент отправляет пакет
Назначение вне локальной сети → пакет идет на шлюз 192.168.10.1

2. Пакет приходит на enp0s8
Это транзитный трафик, не для самого gw01

3. Ядро Linux проверяет маршруты
ip_forward = 1, таблица маршрутизации выбирает выход через enp0s3

4. nftables: chain forward
Firewall решает: разрешить или запретить транзит

5. NAT masquerade: postrouting
Источник меняется: 192.168.10.10 → 203.0.113.10

6. Отправка во внешнюю сеть
Пакет уходит через enp0s3 в Интернет

WAN / Интернет

Интернет / внешний сервер



8.8.8.8

1. Ответ приходит на 203.0.113.10

2. Connection tracking + NAT восстанавливают внутренний адрес клиента

3. Пакет возвращается через enp0s8 к client01

Что должно быть настроено



ip_forward = 1



Маршрут по умолчанию во внешнюю сеть



Разрешающее правило в forward



Masquerade в postrouting



Ключевая идея

Linux-router пересылает пакет между сетями, firewall контролирует транзит, а NAT подменяет внутренний адрес на внешний.

Итог раздела 1

1. Linux может быть маршрутизатором при наличии нескольких интерфейсов.
2. Для транзита нужен net.ipv4.ip_forward=1.
3. Клиенты LAN должны использовать gw01 как default gateway.
4. Маршруты решают, куда отправить пакет.
5. Но для выхода частных адресов наружу одной маршрутизации недостаточно — нужен PAT.

2. NAT и PAT

Как вся LAN выходит наружу через один внешний IP

Зачем нужен NAT

- NAT изменяет IP-адреса в проходящих пакетах.
- Внутренние адреса 192.168.10.0/24 являются частными.
- Такие адреса не маршрутизируются в публичном Интернете.
- gw01 должен заменить внутренний адрес клиента на свой внешний адрес.
- Ответ из внешней сети возвращается на gw01.
- gw01 восстанавливает внутренний адрес клиента и отправляет ответ в LAN.

Почему основная тема — PAT

- PAT — Port Address Translation: вариант NAT, где меняется не только IP-адрес, но и номер порта.
- Один внешний IP может обслуживать много внутренних клиентов.
- gw01 различает соединения по портам.
- Внутренний адрес и порт превращаются во внешний адрес и новый порт.
- PAT часто называют NAT overload.
- Практически это самый привычный сценарий выхода домашней или офисной сети в Интернет.

SNAT и PAT

SNAT

- Меняет адрес источника.
- Может использовать один фиксированный внешний IP.
- Не объясняет, как один IP обслуживает много клиентов.
- Полезен как базовое понятие.

PAT

- Меняет адрес источника и порт.
- Позволяет many-to-one.
- Различает соединения через таблицу трансляций.
- Это главный сценарий лекции.

PAT many-to-one

Один внешний IP для многих внутренних клиентов

LAN / частная сеть

client01
192.168.10.10:49152

web01
192.168.10.20:53001

pc03
192.168.10.30:50110

gw01 Linux-router
PAT / Masquerade

LAN:
192.168.10.1

WAN:
203.0.113.10

Интернет / внешняя сеть

203.0.113.10:40001 → 8.8.8.8:443

203.0.113.10:40002 → 1.1.1.1:443

203.0.113.10:40003 → example.com:80

Таблица трансляций PAT

Внутренний адрес:порт		Внешний адрес:порт
192.168.10.10:49152	→	203.0.113.10:40001
192.168.10.20:53001	→	203.0.113.10:40002
192.168.10.30:50110	→	203.0.113.10:40003

1. Частные адреса LAN не маршрутизируются в Интернет напрямую
2. PAT меняет адрес источника и исходный порт
3. Один внешний IP обслуживает много соединений

Пример PAT-таблицы

Внутри LAN	После PAT на gw01	Назначение
192.168.10.10:49152	203.0.113.10:40001	8.8.8.8:443
192.168.10.20:53001	203.0.113.10:40002	1.1.1.1:443
192.168.10.30:50110	203.0.113.10:40003	example.com:443

Что хранит таблица трансляций

- Внутренний IP клиента.
- Внутренний исходный порт клиента.
- Внешний IP gw01.
- Внешний порт, выбранный для трансляции.
- Адрес и порт внешнего сервера.
- Состояние соединения и время жизни записи.

Путь исходящего пакета при PAT

- client01 отправляет пакет: 192.168.10.10:49152 → 8.8.8.8:443.
- Пакет приходит на gw01 через LAN-интерфейс.
- gw01 выбирает внешний интерфейс enp0s3.
- PAT меняет источник на 203.0.113.10:40001.
- Во внешнюю сеть уходит пакет от имени gw01.
- Таблица трансляций запоминает соответствие портов.

Как возвращается ответ

- Внешний сервер отвечает на 203.0.113.10:40001.
- gw01 ищет порт 40001 в таблице трансляций.
- Находит, что это соединение client01:49152.
- Меняет назначение обратно на 192.168.10.10:49152.
- Отправляет пакет во внутреннюю сеть через enp0s8.

Обратная трансляция PAT

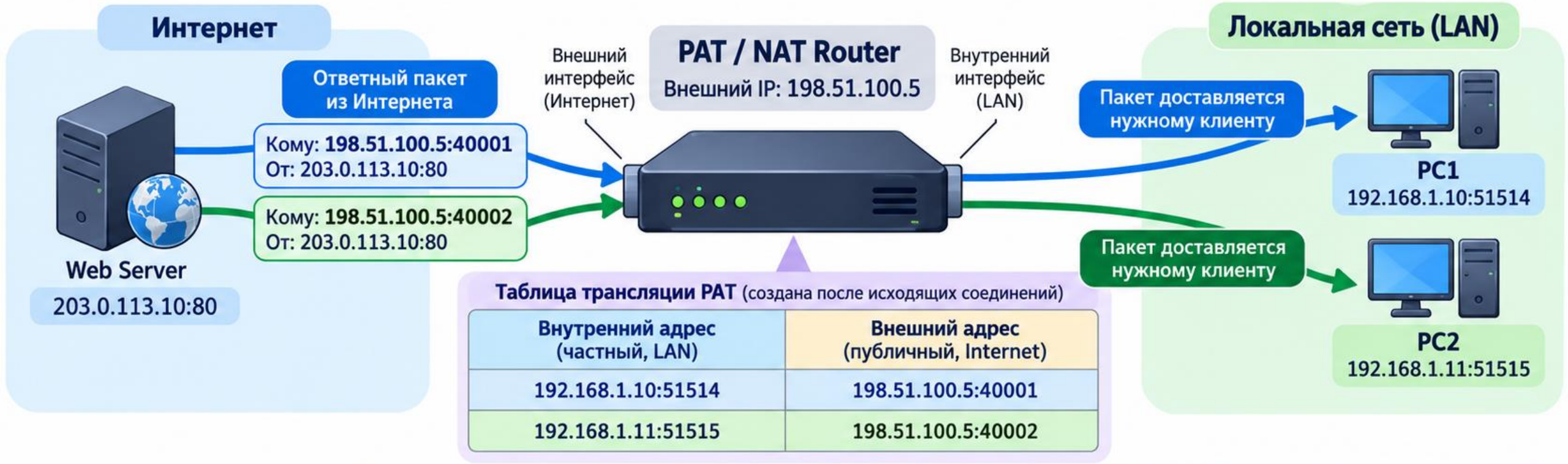


Таблица трансляции PAT (создана после исходящих соединений)

Внутренний адрес (частный, LAN)	Внешний адрес (публичный, Internet)
192.168.1.10:51514	198.51.100.5:40001
192.168.1.11:51515	198.51.100.5:40002

1 Ответный пакет из Интернета

Приходит пакет на 198.51.100.5:40001 или 198.51.100.5:40002 от внешнего сервера.

2 Маршрутизатор ищет запись в таблице PAT

По внешнему IP и порту находит соответствующую запись.

3 Восстановление исходного частного адреса и порта

Выполняется обратная трансляция: 198.51.100.5:40001 → 192.168.1.10:51514 или 198.51.100.5:40002 → 192.168.1.11:51515.

4 Пакет доставляется нужному клиенту

Маршрутизатор отправляет пакет во внутреннюю сеть соответствующему хосту (PC1 или PC2).

PAT позволяет по внешнему IP и номеру порта определить, какому внутреннему хосту принадлежит ответный трафик.

Без записи в таблице PAT входящий пакет не будет сопоставлен с внутренним клиентом.

Masquerade как практический вариант PAT

- Masquerade — динамический вариант исходящего NAT/PAT, при котором внешний адрес берётся с исходящего интерфейса.
- Удобен, если внешний адрес gw01 может меняться.
- Подходит для учебного стенда и типовой edge-схемы.
- Работает на исходящем интерфейсе WAN.
- В нашем стенде исходящий интерфейс — enp0s3.
- Для клиентов LAN это выглядит как обычный доступ наружу.

UFW: включить маршрутизацию и доступ LAN наружу

BASH

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw default deny routed

sudo ufw route allow in on enp0s8 out on enp0s3
sudo ufw enable
sudo ufw status verbose
```

Что важно

- deny routed запрещает транзит по умолчанию.
- ufw route allow разрешает транзит из LAN наружу.
- enp0s8 — LAN, enp0s3 — WAN.
- PAT настраивается отдельно в before.rules.

UFW before.rules: Включаем PAT для LAN

TEXT

```
# /etc/ufw/before.rules
# Блок NAT должен быть ДО строки *filter

*nat
:POSTROUTING ACCEPT [0:0]
-A POSTROUTING -s 192.168.10.0/24 -o enp0s3 -j MASQUERADE
COMMIT
```

Что важно

- Этот блок включает исходящую трансляцию для LAN.
- -s 192.168.10.0/24 выбирает внутреннюю сеть.
- -o enp0s3 указывает внешний интерфейс.
- MASQUERADE подставляет внешний адрес gw01.

Применение UFW после изменения before.rules

BASH

```
sudo ufw disable
sudo ufw enable
sudo ufw status verbose
```

Что важно

- После изменения before.rules UFW нужно перезапустить.
- Проверьте, что UFW включён.
- Затем проверьте выход client01 наружу.
- Если используется удалённое подключение, заранее не потеряйте SSH-доступ.

Проверка выхода LAN через PAT

BASH

```
# На client01
ip route
ping -c 4 192.168.10.1
ping -c 4 8.8.8.8

# На gw01
sysctl net.ipv4.ip_forward
sudo ufw status verbose

# Если pkts увеличивается, значит NAT работает
sudo iptables -t nat -L POSTROUTING -n -v
```

Что важно

- ip route на клиенте должен показывать default via 192.168.10.1.
- ping 192.168.10.1 проверяет доступность gateway.
- ping внешнего IP проверяет маршрутизацию и PAT без DNS.
- ip_forward должен быть равен 1.

Типовые ошибки PAT

- IPv4 forwarding выключен.
- У клиента LAN неверный default gateway.
- PAT указан не для той внутренней сети.
- PAT указан не на том внешнем интерфейсе.
- UFW запрещает routed-трафик.
- Проверку начали с DNS, хотя сначала нужно проверить ping по IP.

Итог раздела 2

1. NAT изменяет адреса в проходящих пакетах.
2. PAT изменяет адрес и порт, поэтому один внешний IP обслуживает много клиентов.
3. gw01 хранит таблицу трансляций для возврата ответов.
4. Masquerade — удобный практический вариант PAT.
5. Для выхода LAN нужны три условия: ip_forward, разрешённый routed-трафик и PAT.

3. Port Forwarding

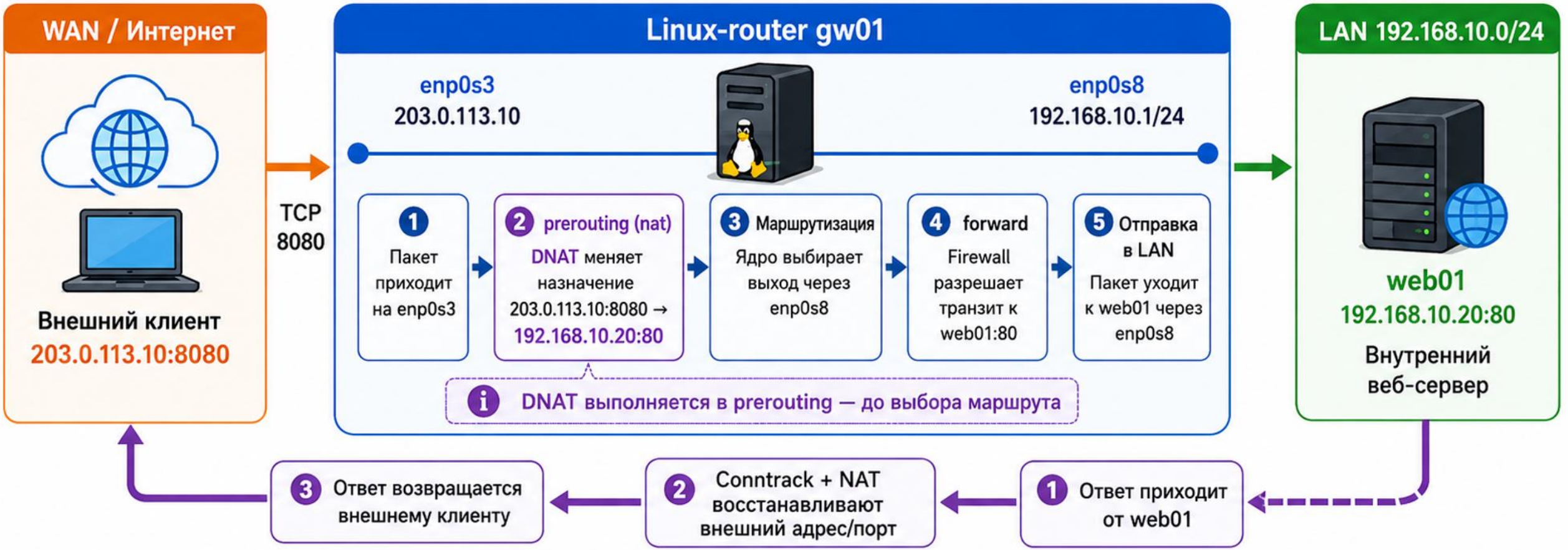
Как опубликовать внутренний web01 через внешний адрес gw01

Задача Port Forwarding

- Port Forwarding публикует внутреннюю службу через внешний адрес и порт маршрутизатора.
- Внутренний web01 слушает TCP 80.
- Внешний клиент подключается к gw01:8080.
- gw01 направляет соединение на web01:80.
- Внешний порт и внутренний порт могут отличаться.
- Публиковать нужно только конкретную службу, а не весь сервер.

Port Forwarding 203.0.113.10:8080 → web01:80

Как Linux-router публикует внутренний веб-сервер через DNAT и forward



Что должно быть настроено

- ip_forward = 1
- DNAT в prerouting
- Правило forward к 192.168.10.20:80
- established,related accept

Ключевая идея

Port Forwarding публикует внутренний сервис наружу: **DNAT** меняет адрес назначения, а **forward** разрешает доставить пакет до **web01**.

DNAT: изменение назначения

- DNAT изменяет адрес или порт назначения во входящем пакете.
- До изменения: внешний клиент → 203.0.113.10:8080.
- После DNAT: внешний клиент → 192.168.10.20:80.
- Изменение нужно выполнить до доставки пакета во внутреннюю сеть.
- DNAT только меняет назначение.
- Для доставки до web01 ещё нужно разрешить транзит через UFW.

Исходящий PAT и входящий Port Forwarding

PAT

- Инициатор — клиент LAN.
- LAN выходит наружу.
- Меняется источник.
- Много клиентов используют один внешний IP.

Port Forwarding

- Инициатор — внешний клиент.
- Внешний запрос идёт внутрь LAN.
- Меняется назначение.
- Публикуется конкретный внутренний сервис.

UFW before.rules: DNAT для web01

TEXT

```
# /etc/ufw/before.rules
# Блок NAT должен быть ДО строки *filter

*nat
:PREROUTING ACCEPT [0:0]
:POSTROUTING ACCEPT [0:0]
-A PREROUTING -i enp0s3 -p tcp --dport 8080 -j DNAT --to-destination 192.168.10.20:80
-A POSTROUTING -s 192.168.10.0/24 -o enp0s3 -j MASQUERADE
COMMIT
```

Что важно

- PREROUTING меняет назначение входящего пакета.
- Внешний TCP 8080 направляется на web01:80.
- POSTROUTING сохраняет PAT для исходящего трафика LAN.
- После изменения UFW нужно перезапустить.

UFW: разрешить транзит к web01:80

BASH

```
sudo ufw route allow in on enp0s3 out on enp0s8 \
  to 192.168.10.20 port 80 proto tcp

sudo ufw reload
sudo ufw status verbose
```

Что важно

- DNAT меняет адрес назначения, но не является разрешающим правилом.
- ufw route allow разрешает доставить пакет к web01.
- Правило узкое: конкретный интерфейс, адрес и порт.
- Так безопаснее, чем разрешать весь транзит снаружи внутрь.

Проверка публикации web01

BASH

```
# На web01
sudo apt install nginx -y
systemctl status nginx

# С внешнего клиента
curl http://203.0.113.10:8080
```

Что важно

- На web01 должна работать служба на TCP 80.
- Внешний клиент обращается к gw01:8080.
- gw01 перенаправляет соединение на web01:80.
- Если ответ не получен, проверяйте DNAT, UFW route и службу web01.

Риски Port Forwarding

- Внутренняя служба становится доступной извне.
- Публикуйте только необходимый порт.
- web01 должен быть обновлён и защищён.
- Логи опубликованной службы нужно отправлять на центральный log-сервер.
- Для административного доступа лучше использовать VPN, а не прямую публикацию SSH.

Итог раздела 3

1. **Port Forwarding решает обратную задачу по сравнению с PAT.**
2. **Внешний порт gw01 направляется на внутренний адрес и порт.**
3. **DNAT меняет назначение пакета.**
4. **UFW route rule разрешает транзит до внутреннего сервиса.**
5. **Публиковать нужно минимально необходимую поверхность.**

4. VPN

Удалённый доступ и маршруты через туннель

Что такое VPN

- VPN создаёт защищённый туннель поверх другой сети, обычно Интернета.
- Удалённый клиент получает адрес внутри VPN-сети.
- Через туннель можно отправлять трафик к корпоративной LAN.
- VPN не отменяет маршруты и правила доступа.
- Для работы нужны адреса, маршруты и разрешения.
- В этой лекции VPN строится через WireGuard.

Site-to-Site и Remote Access

Site-to-Site

- Соединяет две сети.
- Пример: офис и филиал.
- Важны маршруты между подсетями.
- Обычно работает постоянно.

Remote Access

- Подключает отдельного пользователя.
- Пример: сотрудник вне офиса.
- Важен контроль доступных сетей.
- Подходит для ноутбуков и мобильных клиентов.

Что меняется после подключения VPN

- У клиента появляется VPN-адрес, например 10.8.0.2.
- Появляется виртуальный интерфейс туннеля.
- Появляются маршруты к внутренним сетям.
- Трафик к 192.168.10.0/24 может идти через туннель.
- Остальной трафик может идти напрямую или тоже через VPN.

Split tunnel и full tunnel

Split tunnel

- Через VPN идут только корпоративные сети.
- Пример: 192.168.10.0/24.
- Интернет клиента идёт напрямую.
- Меньше нагрузка на VPN-сервер.

Full tunnel

- Через VPN идёт весь трафик.
- Пример: 0.0.0.0/0.
- Интернет клиента тоже проходит через gw01.
- Может потребоваться PAT для VPN-сети.

Remote Access VPN к корпоративной LAN

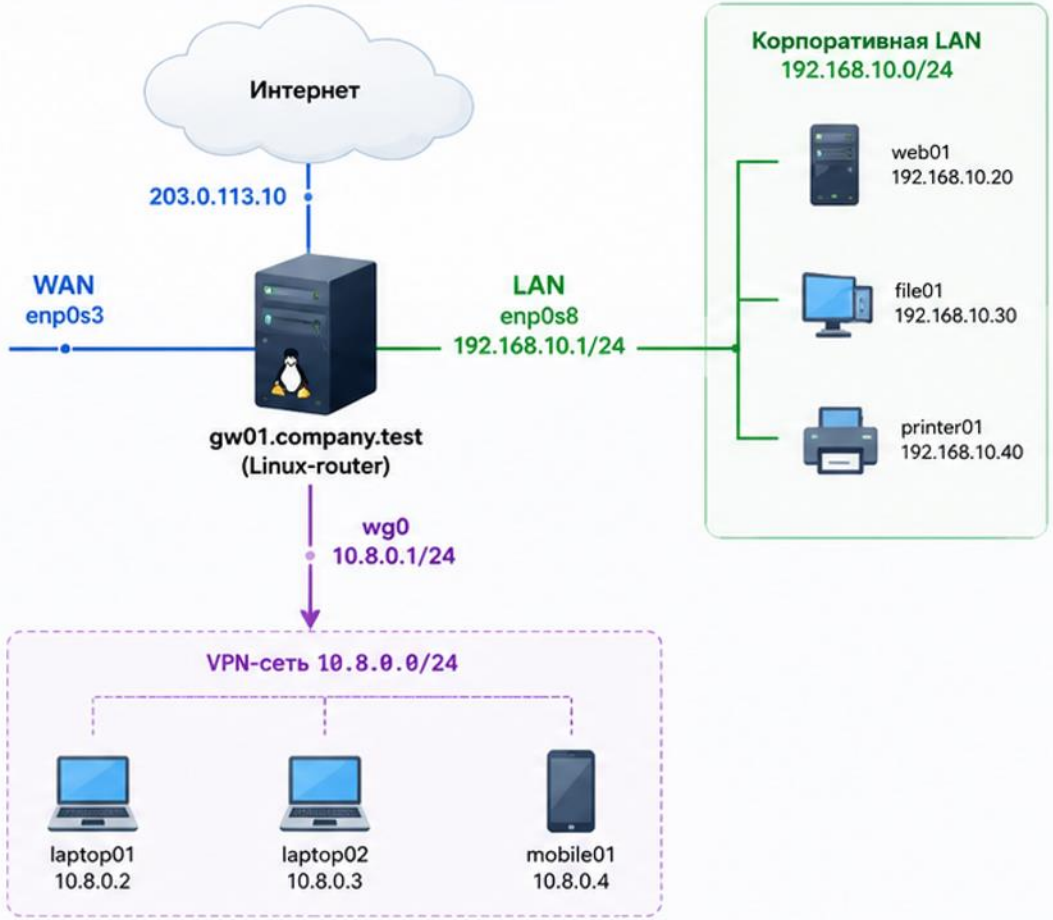
Безопасный удалённый доступ сотрудников к внутренней сети компании через WireGuard VPN

КАК ЭТО РАБОТАЕТ

1. Клиент подключается к VPN-серверу gw01 через Интернет.
2. Устанавливается зашифрованный WireGuard-туннель (wg0). Проверяется ключ клиента.
3. Клиент получает IP из VPN-сети 10.8.0.0/24 и маршрут до LAN 192.168.10.0/24 через gw01.
4. Трафик клиента идёт через туннель на gw01, затем маршрутизируется в корпоративную LAN.
5. Ответы из LAN возвращаются через gw01 в туннель и приходят к клиенту.

ПРИМЕР ПОДКЛЮЧЕНИЯ





ПРЕИМУЩЕСТВА

Шифрование трафика
WireGuard использует современные алгоритмы (Noise IK, ChaCha20, Poly1305).

Простой удалённый доступ
Сотрудники работают как будто в офисной сети, без проброса портов.

Маршрутизация через VPN
Доступ только к нужным сетям (например, 192.168.10.0/24).

Высокая производительность
Минимальные накладные расходы и быстрая установка соединения.

ПРИМЕР КОНФИГУРАЦИИ (КРАТКО)

Сервер (gw01) – /etc/wireguard/wg0.conf

```
[Interface]
Address = 10.8.0.1/24
ListenPort = 51820
PrivateKey = <server_private_key>

# Разрешить маршрутизацию и доступ в LAN
PostUp = sysctl -w net.ipv4.ip_forward=1
PostUp = nft add rule inet filter forward iifname "wg0" oifname "enp0s8" accept
PostUp = nft add rule inet filter forward iifname "enp0s8" oifname "wg0" ct
         state related,established accept
# Маршрут к LAN уже есть: 192.168.10.0/24 dev enp0s8
```

Клиент (laptop01) – /etc/wireguard/wg0.conf

```
[Interface]
Address = 10.8.0.2/24
PrivateKey = <client_private_key>
DNS = 192.168.10.1

[Peer]
PublicKey = <server_public_key>
Endpoint = 203.0.113.10:51820
AllowedIPs = 192.168.10.0/24, 10.8.0.0/24
PersistentKeepalive = 25
```

AllowedIPs определяет, какие сети пойдут через VPN.

ЧТО НУЖНО НА GW01

- Включить маршрутизацию IPv4: `sysctl -w net.ipv4.ip_forward=1`
- Разрешить форвард между wg0 и LAN (правила forward, как в примере выше)
- Открыть в firewall (nftables): UDP порт 51820 для WireGuard
- Маршрут к LAN (192.168.10.0/24) должен быть в таблице маршрутизации

ПРОВЕРКА РАБОТЫ

```
# На клиенте
$ wg show                          # пинг VPN-сервера
$ ping 10.8.0.1                    # пинг хоста в LAN
$ tracert²192.168.10.20            # маршрут должен идти через 10.8.0.1

# На сервере
d$ wg show                         # список подключенных пиров
d$ ip route | grep 10.8.0.         # маршрут до VPN-сети
$ nft list ruleset | grep wg0      # правила firewall
```

БЕЗОПАСНОСТЬ

- Используйте уникальные ключи для каждого клиента.
- Разрешайте доступ только к необходимым сетям (AllowedIPs).
- Следите за журналами: `journalctl -u wg-quick@wg0`
- Регулярно обновляйте систему и проверяйте конфигурацию.

ИТОГ

WireGuard VPN позволяет сотрудникам безопасно подключаться к ресурсам корпоративной LAN из любой точки мира.

Итог раздела 4

1. VPN — это не только шифрование, но и маршрутизация.
2. Remote Access подключает отдельного пользователя к внутренней сети.
3. Клиенту нужен VPN-адрес и маршрут к LAN.
4. Split tunnel отправляет через VPN только нужные сети.
5. Full tunnel отправляет через VPN весь трафик клиента.

5. WireGuard

Peers, ключи, Endpoint, AllowedIPs и handshake

Что такое WireGuard

- WireGuard — современный минималистичный VPN, который использует пары криптографических ключей.
- Работает поверх UDP.
- Часто использует порт 51820.
- Каждый участник туннеля называется peer.
- У каждого peer есть private key и public key.
- Настройка компактная, но требует точного понимания AllowedIPs.

Peer

- Peer — участник WireGuard-туннеля.
- Сервер gw01 является peer.
- Удалённый ноутбук laptop01 тоже является peer.
- Каждый peer знает свой private key.
- Каждый peer получает public key другой стороны.
- WireGuard не использует логины и пароли для установления туннеля.

Private key и public key

- Private key хранится только на своём узле.
- Private key нельзя отправлять в чат, отчёт или общую папку.
- Public key можно передавать другой стороне.
- Сервер хранит public key клиента.
- Клиент хранит public key сервера.
- Если private key утёк, пару ключей нужно заменить.

Генерация ключей WireGuard

BASH

```
umask 077
wg genkey | tee private.key | wg pubkey > public.key

cat private.key
cat public.key
```

Что важно

- umask 077 ограничивает права на создаваемые файлы.
- private.key остаётся на своём узле.
- public.key передаётся другой стороне.
- Такую пару нужно создать на сервере и на клиенте.

Endpoint

- Endpoint отвечает на вопрос: куда физически отправлять зашифрованные WireGuard-пакеты.
- На клиенте Endpoint обычно указывает публичный адрес сервера.
- Пример: Endpoint = 203.0.113.10:51820.
- Это адрес и UDP-порт, куда клиент отправляет VPN-пакеты.
- Если Endpoint неверный, handshake не появится.
- У сервера Endpoint клиента может определяться динамически после первого пакета.

AllowedIPs

- AllowedIPs связывает peer с IP-адресами и маршрутами.
- На сервере AllowedIPs = 10.8.0.2/32 закрепляет VPN-адрес клиента за его public key.
- На клиенте AllowedIPs = 192.168.10.0/24 означает: отправлять корпоративную LAN через туннель.
- Для full tunnel на клиенте используют AllowedIPs = 0.0.0.0/0.
- Слишком широкие AllowedIPs могут увести трафик не туда.
- Слишком узкие AllowedIPs могут не создать нужный маршрут.

Endpoint и AllowedIPs

Endpoint

- Куда отправлять VPN-пакеты.
- Обычно внешний IP и UDP-порт сервера.
- Пример: 203.0.113.10:51820.
- Ошибка Endpoint → нет handshake.

AllowedIPs

- Какие обычные IP-сети идут через peer.
- На сервере закрепляет адрес клиента.
- На клиенте создаёт маршруты.
- Ошибка AllowedIPs → туннель есть, но сеть не доступна.

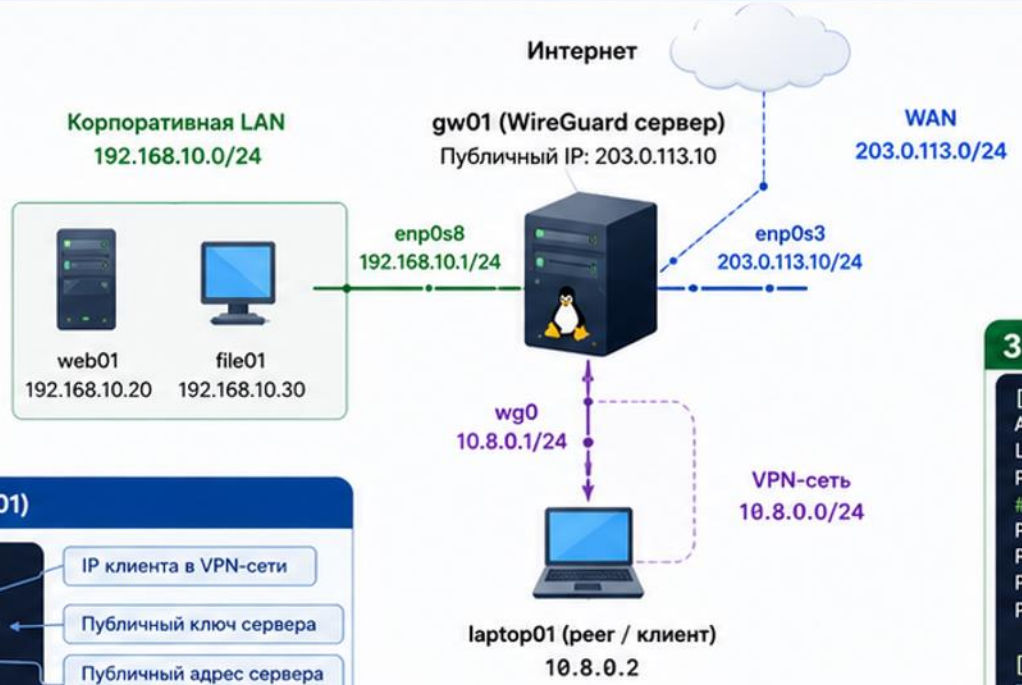
WireGuard: peers, ключи и AllowedIPs

Как клиенты (peers) устанавливают защищённый туннель и какие сети они могут маршрутизировать

1 Что такое peer?

Peer (пир) — это удалённый узел WireGuard, с которым устанавливается VPN-туннель.

Каждый peer имеет свою пару криптографических ключей и обменивается публичными ключами для безопасного соединения.



2 Криптографические ключи

Каждый узел генерирует пару ключей:

- **PrivateKey** — приватный ключ (хранится в секрете)
- **PublicKey** — публичный ключ (обменивается с peers)

Приватный ключ → однонаправленно → Публичный ключ

- Приватный ключ никогда не передаётся по сети.
- Публичные ключи peers указываются в конфигурации.
- WireGuard использует современные алгоритмы (NoiseIK, Curve25519, ChaCha20).

3 Конфигурация сервера (gw01)

```
[Interface]
Address = 10.8.0.1/24          ← Интерфейс wg0 на сервере
ListenPert = 51820
PrivateKey = <SERVER_PRIVATE_KEY>
# Включаем маршрутизацию и NAT для LAN
PostUp = sysctl -w net.ipv4.ip_forward=1
PostUp = nft add rule inet filter forward iifname "wg0" oifname "enp0s8" accept
PostUp = nft add rule ip nat postrouting oifname "enp0s3" masquerade
PostDown = nft flush ruleset

[Peer]
PublicKey = <LAPTOP01_PUBLIC_KEY>      ← Публичный ключ клиента
AllowedIPs = 10.8.0.2/32, 192.168.10.0/24   ← Разрешённые сети клиента: его IP в VPN и доступ к LAN
PersistentKeepalive = 25
```

4 Конфигурация клиента (laptop01)

```
[Interface]
Address = 10.8.0.2/24          ← IP клиента в VPN-сети
PrivateKey = <LAPTOP01_PRIVATE_KEY>   ← Публичный ключ сервера
DNS = 192.168.10.1             ← Публичный адрес сервера и порт WireGuard

[Peer]
PublicKey = <SERVER_PUBLIC_KEY>
Endpoint = 203.0.113.10:51820
AllowedIPs = 10.8.0.0/24, 192.168.10.0/24   ← Какие сети доступны через VPN (вся VPN-сеть и корпоративная LAN)
PersistentKeepalive = 25       ← Поддержание соединения за NAT/фаерволом
```

5 AllowedIPs — что это и как работает

AllowedIPs определяет какие IP-сети «принадлежат» данному peer и будут отправляться в туннель.

Если сети нет в AllowedIPs — трафик пойдёт в обычный интернет, а не в туннель.

Пример маршрутизации на клиенте

Назначение	Куда отправить
10.8.0.0/24	через wg0 (VPN)
192.168.10.0/24	через wg0 (VPN)
8.8.8.8	через обычный интернет
0.0.0.0/0	через обычный интернет

Совет: указывайте только нужные сети, чтобы не туннелировать лишний трафик.

6 Полезные команды

```
# Показать интерфейсы WireGuard
wg show

# Показать публичный ключ
wg show wg0 public-key

# Проверить маршрут к сети LAN
ip route get 192.168.10.20

# Пинг через VPN
ping 10.8.0.1

# Пинг узла в LAN через туннель
ping 192.168.10.20

# Логи ядра (при проблемах)
journalctl -k | grep wireguard
```

Кратко

- Каждый peer имеет ключи: **PrivateKey** (секрет) и **PublicKey** (для обмена).
- Peers обмениваются публичными ключами и устанавливают защищённый туннель.
- **AllowedIPs** управляет тем, какие сети доступны через этот туннель.
- Комбинация wg0 + ip_forward + NAT позволяет безопасно подключаться к LAN из любой точки мира.

Важно

- Храните приватные ключи в безопасности.
- Используйте минимально необходимые сети в AllowedIPs.
- Регулярно обновляйте систему и следите за логами.

Handshake

- Handshake подтверждает, что peers обменялись защищёнными WireGuard-пакетами.
- Команда wg show показывает latest handshake.
- Если handshake отсутствует, туннель фактически не работает.
- Причины: UDP 51820 закрыт, неверный Endpoint, перепутаны ключи.
- Если handshake есть, но LAN недоступна, проблема чаще в маршрутах, AllowedIPs, ip_forward или UFW.
- Счётчики transfer помогают понять, идут ли данные.

WireGuard и UFW

- На внешнем интерфейсе gw01 нужно разрешить UDP 51820.
- Между wg0 и LAN нужно разрешить маршрутизацию.
- Если VPN-клиент должен выходить в Интернет через gw01, для сети 10.8.0.0/24 нужен PAT.
- WireGuard создаёт туннель, но не заменяет маршрутизацию и правила доступа.

Итог раздела 5

1. **WireGuard строится на peers и парах ключей.**
2. **Private key никогда не передаётся.**
3. **Endpoint отвечает за физический адрес peer.**
4. **AllowedIPs отвечает за маршруты и привязку адресов к peer.**
5. **Handshake показывает, что peers действительно обмениваются VPN-пакетами.**

6. Практика WireGuard

Сервер gw01 и удалённый клиент laptop01

Задача практики

- gw01 работает как WireGuard-сервер.
- laptop01 работает как удалённый клиент.
- gw01 получает адрес 10.8.0.1/24 на wg0.
- laptop01 получает адрес 10.8.0.2/32.
- laptop01 должен получить доступ к LAN 192.168.10.0/24.
- Доступ к LAN проходит через gw01.

Общий порядок настройки

1	**Шаг 1**	Установить WireGuard на сервере и клиенте.
2	**Шаг 2**	Создать ключи сервера.
3	**Шаг 3**	Создать ключи клиента.
4	**Шаг 4**	Обменяться только public key.
5	**Шаг 5**	Создать /etc/wireguard/wg0.conf на сервере.
6	**Шаг 6**	Создать конфигурацию клиента.
7	**Шаг 7**	Разрешить UDP 51820 через UFW.
8	**Шаг 8**	Разрешить routed-трафик между wg0 и LAN.
9	**Шаг 9**	Запустить wg0.
10	**Шаг 10**	Проверить wg show, ping 10.8.0.1 и доступ к 192.168.10.20.

Установка WireGuard

BASH

```
sudo apt update
sudo apt install wireguard -y

wg --version
```

Что важно

- Пакет wireguard устанавливает wg и wg-quick.
- wg показывает состояние и ключи.
- wg-quick поднимает интерфейс по файлу конфигурации.
- Команды выполняются на gw01 и на клиенте Linux.

Ключи сервера gw01

BASH

```
sudo mkdir -p /etc/wireguard
cd /etc/wireguard
sudo sh -c 'umask 077; wg genkey | tee server_private.key | wg pubkey > server_public.key'

sudo cat server_public.key
```

Что важно

- server_private.key остаётся на gw01.
- server_public.key передаётся клиенту.
- umask 077 защищает файлы ключей.
- Закрытый ключ сервера нельзя вставлять в отчёт.

Ключи клиента laptop01

BASH

```
umask 077
wg genkey | tee client_private.key | wg pubkey > client_public.key

cat client_public.key
```

Что важно

- client_private.key остаётся на клиенте.
- client_public.key передаётся на сервер.
- На сервере этот public key будет указан в секции [Peer].
- Если ключи перепутаны, handshake не появится.

Конфигурация сервера wg0.conf

```ini
[Interface]
Address = 10.8.0.1/24
ListenPort = 51820
PrivateKey = <SERVER_PRIVATE_KEY>

[Peer]
PublicKey = <CLIENT_PUBLIC_KEY>
AllowedIPs = 10.8.0.2/32
```

Что важно

- Address задаёт адрес сервера внутри VPN.
- ListenPort — UDP-порт WireGuard.
- PrivateKey — закрытый ключ сервера.
- PublicKey в секции Peer — открытый ключ клиента.
- AllowedIPs = 10.8.0.2/32 закрепляет VPN-адрес клиента.

Права на конфигурацию сервера

BASH

```
sudo chmod 600 /etc/wireguard/wg0.conf
sudo chown root:root /etc/wireguard/wg0.conf
```

Что важно

- wg0.conf содержит private key сервера.
- Файл должен читать только root.
- Слишком открытые права — риск утечки ключа.
- При утечке private key нужно заменить ключи.

UFW: разрешить WireGuard и доступ в LAN

BASH

```
sudo ufw allow in on enp0s3 proto udp to any port 51820
sudo ufw route allow in on wg0 out on enp0s8 to 192.168.10.0/24
sudo ufw reload
sudo ufw status verbose
```

Что важно

- UDP 51820 нужен для входящих WireGuard-пакетов.
- Правило route разрешает VPN-клиенту идти в LAN.
- wg0 — VPN-интерфейс.
- enp0s8 — LAN-интерфейс.

Запуск WireGuard на сервере

BASH

```
sudo wg-quick up wg0
sudo wg
ip addr show wg0

sudo systemctl enable wg-quick@wg0
```

Что важно

- wg-quick up wg0 поднимает интерфейс.
- wg показывает состояние WireGuard.
- ip addr подтверждает адрес 10.8.0.1/24.
- enable включает автозапуск wg0.

Конфигурация клиента

```ini
[Interface]
Address = 10.8.0.2/32
PrivateKey = <CLIENT_PRIVATE_KEY>

[Peer]
PublicKey = <SERVER_PUBLIC_KEY>
Endpoint = 203.0.113.10:51820
AllowedIPs = 192.168.10.0/24
PersistentKeepalive = 25
```

Что важно

- Address — VPN-адрес клиента.
- Endpoint — внешний адрес сервера.
- AllowedIPs = 192.168.10.0/24 создаёт split tunnel к LAN.
- PersistentKeepalive полезен, если клиент находится за NAT.

Проверка клиента

```bash
sudo wg-quick up wg0
sudo wg
ip route
ping -c 4 10.8.0.1
ping -c 4 192.168.10.20
```

Что важно

- ping 10.8.0.1 проверяет VPN-интерфейс сервера.
- ping 192.168.10.20 проверяет доступ к LAN.
- ip route должен показать маршрут к 192.168.10.0/24 через wg0.
- Если handshake есть, но LAN недоступна, проверяйте ip_forward, UFW route и AllowedIPs.

Если нужен full tunnel

- На клиенте AllowedIPs меняют на 0.0.0.0/0.
- Тогда весь IPv4-трафик клиента идёт через VPN.
- На gw01 нужно разрешить routed-трафик от wg0 к WAN.
- Для сети 10.8.0.0/24 нужен PAT наружу.
- Full tunnel увеличивает нагрузку на gw01.

Дополнительный PAT для VPN-сети

```text
# /etc/ufw/before.rules
# В блок *nat добавить строку:
-A POSTROUTING -s 10.8.0.0/24 -o enp0s3 -j MASQUERADE
```

Что важно

- Это нужно только если VPN-клиенты выходят в Интернет через gw01.
- 10.8.0.0/24 — VPN-сеть.
- enp0s3 — WAN-интерфейс.
- После изменения перезапустите UFW.

Итог практики WireGuard

1. Сервер и клиент имеют пары ключей.
2. Стороны обмениваются только public key.
3. Endpoint задаёт адрес сервера для клиента.
4. AllowedIPs определяет маршруты через туннель.
5. UFW должен разрешить UDP 51820 и routed-трафик между wg0 и LAN.
6. wg show и ping помогают проверить результат.

7. Диагностика

Проверяем по слоям: маршруты → PAT → UFW → WireGuard

Главный принцип диагностики

- Не переписывайте всю конфигурацию сразу. Сначала определите, на каком уровне проблема.
- Интерфейсы и адреса.
- Таблица маршрутизации.
- IPv4 forwarding.
- UFW routed rules.
- PAT и Port Forwarding.
- WireGuard handshake.
- AllowedIPs и маршруты клиента.

Диагностика Linux-router, NAT и WireGuard

Пошаговая проверка маршрутизации, firewall, NAT и VPN в учебном стенде

1. Базовая проверка интерфейсов

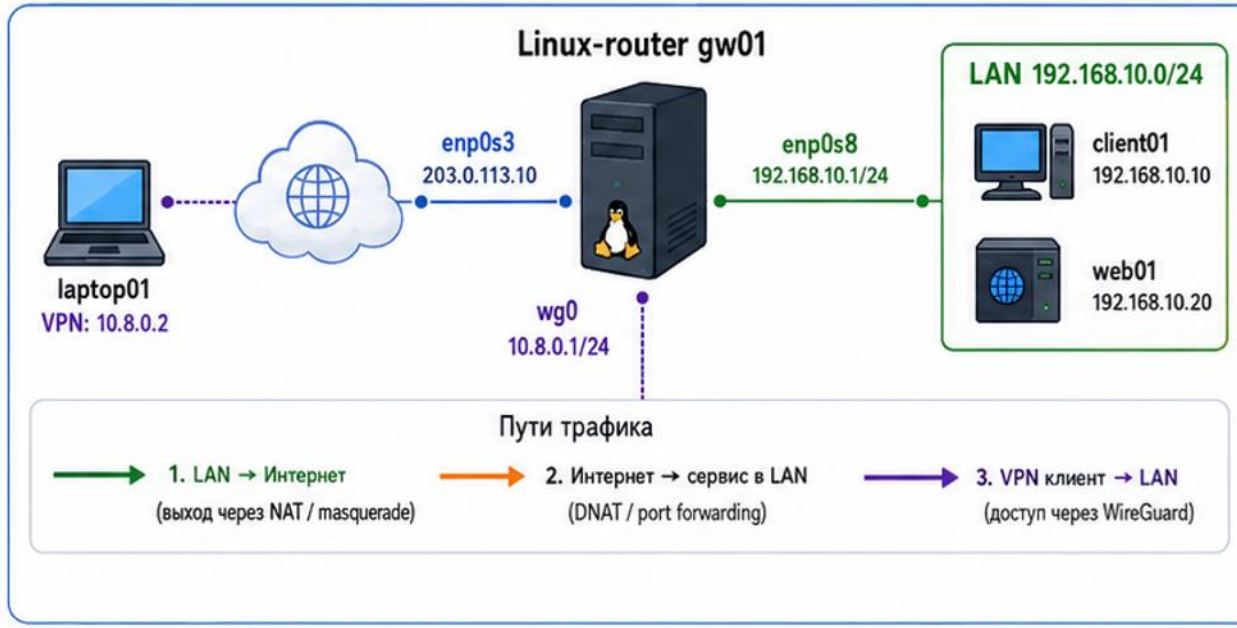
- `ip a` — проверить адреса enp0s3, enp0s8, wg0
- `ip route` — проверить default route и локальные сети
- `ping 192.168.10.1` — доступность шлюза

Если интерфейс down или адрес неверный, дальше проверять рано.

2. Проверка маршрутизации Linux-router

- `sysctl net.ipv4.ip_forward` → должно быть 1
- Проверить маршрут в Интернет через enp0s3
- Проверить маршрут к 192.168.10.0/24 и 10.8.0.0/24

ip_forward=0 — транзитный трафик не пойдёт



3. Проверка nftables / firewall

- `sudo nft list ruleset`
- Цепочка **input:** доступ к SSH / WireGuard / локальным службам
- Цепочка **forward:** разрешение LAN↔WAN и wg0↔LAN
- policy drop требует явных accept-правил

prerouting (DNAT) → input (к роутеру) → forward (транзит) → postrouting (NAT)

4. Проверка NAT и port forwarding

A) Masquerade для выхода LAN наружу

- правило masquerade в postrouting для oifname "enp0s3"
- клиент LAN должен иметь gateway 192.168.10.1

B) DNAT / Port Forwarding

- prerouting dnat 203.0.113.10:8080 -> 192.168.10.20:80
- forward должен разрешать трафик к web01

DNAT без forward не работает

5. Проверка WireGuard

- `wg show` — peer, latest handshake, transfer
- На gw01 разрешён UDP 51820
- AllowedIPs заданы корректно
- Должен быть разрешён forward между wg0 и enp0s8

Туннель поднят, но LAN не видна? Проверить ip_forward, AllowedIPs и forward.

6. Практические тесты

- [x] С client01: ping 8.8.8.8
- [x] С client01: ping внешнего IP / проверка выхода через NAT
- [x] С внешнего клиента: доступ к 203.0.113.10:8080
- [x] С laptop01: ping 10.8.0.1
- [x] С laptop01: ping 192.168.10.20
- [x] traceroute / tracepath до нужного узла

7. Полезные команды

```
$ ip a
$ ip route
$ sysctl net.ipv4.ip_forward
$ sudo nft list ruleset
$ wg show
$ ping 8.8.8.8
$ traceroute 192.168.10.20
$ sudo journalctl -u wg-quick@wg0
$ sudo tcpdump -ni enp0s3
```

Типовые симптомы и причины

Нет Интернета у LAN
- gateway
- ip_forward
- masquerade
- forward

Порт 8080 снаружи не открывается
- DNAT
- forward
- сервис web01

VPN handshake нет
- UDP 51820
- ключи
- endpoint

VPN есть, но LAN не доступна
- AllowedIPs
- forward wg0↔enp0s8
- маршруты

Ключевая идея

Диагностика идёт по цепочке: интерфейсы -> маршруты -> ip_forward -> nftables -> NAT -> WireGuard -> тестовый трафик и логи.

Проверка Linux-router

```bash
ip addr
ip route
sysctl net.ipv4.ip_forward
ping -c 4 192.168.10.1
```

Что важно

- ip addr показывает адреса интерфейсов.
- ip route показывает маршруты.
- ip_forward должен быть равен 1.
- ping 192.168.10.1 с LAN-клиента проверяет доступность gateway.

Проверка PAT

```bash
# На client01
ip route
ping -c 4 192.168.10.1
ping -c 4 8.8.8.8

# На gw01
sudo ufw status verbose
```

Что важно

- Сначала проверяем gateway.
- Затем внешний IP без DNS.
- Если ping gateway работает, а внешний IP нет — смотрим ip_forward, PAT и UFW.
- Если внешний IP работает, а домены нет — проблема уже в DNS.

Симптом: у LAN нет Интернета

- Проверить default gateway на client01.
- Проверить ip_forward на gw01.
- Проверить UFW route allow из LAN в WAN.
- Проверить PAT в before.rules.
- Проверить внешний default route на gw01.
- Проверить ping по IP, а не сразу по DNS-имени.

Симптом: Port Forwarding не работает

- Проверить, работает ли web01 на внутреннем порту 80.
- Проверить, доступен ли web01:80 с gw01.
- Проверить DNAT в before.rules.
- Проверить UFW route allow к 192.168.10.20:80.
- Проверить, что внешний клиент обращается именно к 203.0.113.10:8080.
- Проверить логи web01.

Проверка WireGuard

```bash
sudo wg
ip addr show wg0
ip route
ping -c 4 10.8.0.1
ping -c 4 192.168.10.20
```

Что важно

- wg показывает latest handshake и transfer.
- ip addr show wg0 проверяет VPN-адрес.
- ip route показывает маршруты через wg0.
- ping 10.8.0.1 проверяет туннель.
- ping 192.168.10.20 проверяет доступ к LAN.

Симптом: нет WireGuard handshake

- Проверить Endpoint на клиенте.
- Проверить внешний IP и UDP-порт сервера.
- Проверить UFW allow UDP 51820 на gw01.
- Проверить, не перепутаны ли private key и public key.
- Проверить, что серверный wg0 поднят.
- Проверить системное время, если есть подозрение на странное поведение.

Симптом: handshake есть, но LAN недоступна

- Проверить AllowedIPs на клиенте: есть ли 192.168.10.0/24.
- Проверить ip_forward на gw01.
- Проверить UFW route allow между wg0 и LAN.
- Проверить, что web01 имеет правильный gateway.
- Проверить ping 10.8.0.1 отдельно от ping 192.168.10.20.
- Разделить проблему: туннель работает или маршрутизация за туннелем.

Карта симптомов

Симптом	Вероятная зона проблемы
client01 не пингует 192.168.10.1	LAN-адресация или интерфейс gw01
client01 пингует gateway, но не 8.8.8.8	ip_forward, PAT, UFW route, default route gw01
Внешний клиент не открывает 203.0.113.10:8080	DNAT, UFW route, служба web01
WireGuard handshake отсутствует	Endpoint, UDP 51820, ключи, UFW
Handshake есть, но 192.168.10.20 не пингуется	AllowedIPs, ip_forward, UFW route, gateway web01
VPN-клиент видит LAN, но не Интернет	Full tunnel, маршрут 0.0.0.0/0, PAT для 10.8.0.0/24

Итоги лекции

1. Linux-router пересылает пакеты между интерфейсами при включённом ip_forward.
2. PAT позволяет всей LAN использовать один внешний IP за счёт трансляции портов.
3. Port Forwarding публикует конкретный внутренний сервис наружу.
4. VPN даёт удалённому клиенту маршрут к корпоративной сети.
5. WireGuard строится на ключах, peers, Endpoint и AllowedIPs.
6. UFW управляет разрешениями для входящего и транзитного трафика.
7. Диагностика должна идти по слоям: интерфейсы, маршруты, forwarding, PAT, UFW, VPN.

Контрольные вопросы

1. Какая переменная ядра включает пересылку IPv4-пакетов между интерфейсами?
2. Чем default gateway отличается от локального маршрута?
3. Почему частная сеть 192.168.10.0/24 не может напрямую выходить в Интернет?
4. Что меняет PAT в исходящем пакете?
5. Как один внешний IP может обслуживать несколько внутренних клиентов?
6. Что хранится в таблице трансляций PAT?
7. Чем исходящий PAT отличается от Port Forwarding?
8. Зачем при Port Forwarding нужно не только изменение назначения, но и разрешение транзита?
9. Что такое Remote Access VPN?
10. Чем Endpoint отличается от AllowedIPs в WireGuard?
11. Что показывает latest handshake в wg show?
12. Что проверить, если WireGuard подключился, но LAN недоступна?